

# Desenvolvimento Web

## Guia de Estudos — Parte 2: Rotas e Navegação

Este material continua o guia anterior (que foi até o projeto Loja Tech) e cobre o novo assunto: rotas. Aqui você vai entender como uma aplicação React mostra várias "páginas" diferentes sem nunca recarregar o navegador, e como isso se aplica ao catálogo de filmes que estamos construindo.

React Router • BrowserRouter • Rotas com Parâmetro • Navegação Programática

# Sumário

---

1. **Por que precisamos de rotas** — o problema que elas resolvem
2. **Instalando e configurando o React Router** — BrowserRouter, Routes, Route
3. **Link** — navegando sem recarregar a página
4. **Montando um projeto com várias páginas** — exemplo completo
5. **Rotas com parâmetro** — `useParams`
6. **Navegação programática** — `useNavigate`
7. **Página 404** — a rota coringa

# 1. Por que precisamos de rotas

---

O problema que rotas resolvem numa aplicação React.

Até agora, todos os projetos que fizemos tinham **uma única tela**. Tudo que existia estava dentro do mesmo `App`. Mas uma aplicação de verdade quase sempre tem várias telas: Início, Sobre, Contato, a página de detalhe de um produto específico. O desafio é: como mostrar telas diferentes **sem recarregar a página inteira** — que é justamente o que o React inteiro está tentando evitar desde a primeira aula?

A resposta é uma **Single Page Application** (SPA) — uma aplicação de página única. Na prática, existe sempre **um único HTML** carregado (o `index.html` do projeto), e o React troca o que aparece na tela dependendo do endereço (URL) que está no navegador, sem nunca recarregar de verdade.

## Analogia

Pense num apartamento com várias salas. Recarregar a página seria como demolir o prédio inteiro e reconstruir só para trocar de cômodo. Uma SPA com rotas é como abrir e fechar portas dentro do mesmo apartamento — você muda de cômodo (de tela), mas o prédio (a página) nunca é reconstruído.

**Rotas** são a ferramenta que liga um endereço (por exemplo, `/sobre`) a um componente específico que deve aparecer na tela quando esse endereço estiver ativo. Quem faz isso no React é uma biblioteca externa chamada **React Router** (pacote `react-router-dom`) — ela não vem embutida no React, por isso instalamos separadamente.

## 2. Instalando e Configurando o React Router

As três peças principais: BrowserRouter, Routes e Route.

### Instalação

```
npm install react-router-dom
```

Isso baixa a biblioteca e adiciona ela ao `package.json`, do mesmo jeito que qualquer outra dependência do projeto.

### As três peças

```
import { BrowserRouter, Routes, Route } from "react-router-dom";
```

| Peça          | Para que serve                                                                                        |
|---------------|-------------------------------------------------------------------------------------------------------|
| BrowserRouter | "Liga" o sistema de rotas — precisa envolver toda a aplicação, uma única vez, no nível mais alto.     |
| Routes        | Um "porteiro" que olha o endereço atual e decide qual <code>Route</code> bate com ele.                |
| Route         | Cada rota individual: liga um caminho ( <code>path</code> ) a um componente ( <code>element</code> ). |

```
<BrowserRouter>
  <Routes>
    <Route path="/" element={<Home />} />
    <Route path="/sobre" element={<Sobre />} />
  </Routes>
</BrowserRouter>
```

Leia assim: "quando o endereço for `/`, mostre o componente `Home`; quando for `/sobre`, mostre o `Sobre`". Repare que `Home` e `Sobre` são só **componentes comuns** — nada diferente do que já fazíamos; a novidade é só como eles são "ligados" a um endereço.

### 3. `Link` — Navegando sem Recarregar

---

Por que não usar a tag `<a>` comum.

Para navegar entre rotas, **não use** a tag `<a href="...">` comum. Ela recarrega a página inteira, o que anula toda a vantagem de uma SPA. Use o componente `Link`, do próprio React Router:

```
import { Link } from "react-router-dom";

<Link to="/sobre">Sobre</Link>
```

#### Erro comum

Usar `<a href="/sobre">` em vez de `<Link to="/sobre">`. Visualmente parece igual, mas por baixo dos panos o `<a>` recarrega a página inteira (perde qualquer estado que existisse, refaz todo o carregamento), enquanto o `Link` só troca o conteúdo, mantendo a experiência de SPA.

## 4. Montando um Projeto com Várias Páginas

Juntando BrowserRouter, Routes, Route e Link num exemplo completo.

Estrutura de pastas sugerida:

```
src/  
├─ App.jsx  
└─ pages/  
    ├─ Home.jsx  
    ├─ Sobre.jsx  
    └─ Contato.jsx
```

```
// src/pages/Home.jsx  
export default function Home() {  
  return <h1>Página Inicial</h1>;  
}
```

```
// src/pages/Sobre.jsx  
export default function Sobre() {  
  return <h1>Sobre nós</h1>;  
}
```

```
// src/App.jsx  
import { BrowserRouter, Routes, Route, Link } from "react-router-dom";  
import Home from "../pages/Home";  
import Sobre from "../pages/Sobre";  
import Contato from "../pages/Contato";  
  
export default function App() {  
  return (  
    <BrowserRouter>  
      <nav>  
        <Link to="/">Início</Link>  
        <Link to="/sobre">Sobre</Link>  
        <Link to="/contato">Contato</Link>  
      </nav>  
  
      <Routes>  
        <Route path="/" element={<Home />} />  
        <Route path="/sobre" element={<Sobre />} />  
        <Route path="/contato" element={<Contato />} />  
      </Routes>  
    </BrowserRouter>  
  );  
}
```

## Explicando bloco a bloco

- `<BrowserRouter>` envolve **tudo** — é ele quem ativa o sistema de rotas na aplicação inteira.
- O `<nav>` com os `<Link>` fica **fora** do `<Routes>` — assim o menu aparece em **todas** as páginas, e não some quando você navega para outra rota.
- `<Routes>` contém as `<Route>` — só uma delas fica visível por vez, dependendo do endereço atual do navegador.

### Erro comum

Colocar o menu ( `<nav>` ) **dentro** do `<Routes>` por engano — aí ele só aparece na rota que o contém, em vez de aparecer em todas as páginas.

## 5. Rotas com Parâmetro

Uma única rota para representar vários itens diferentes.

Até aqui, cada rota tinha um caminho **fixo** ( `/sobre` , `/contato` ). Mas e se você quiser uma página de detalhe para **cada filme** do catálogo, sem escrever uma rota para cada um? A resposta é uma rota com **parâmetro dinâmico**, marcado com `:` no caminho:

```
<Route path="/filme/:id" element={<FilmeDetalhe />} />
```

O `:id` é uma "variável" dentro do endereço. Tanto `/filme/1` quanto `/filme/42` batem com essa mesma rota — o valor depois de `/filme/` fica disponível dentro do componente.

### Lendo o parâmetro com `useParams`

```
import { useParams } from "react-router-dom";

function FilmeDetalhe() {
  const { id } = useParams();

  return <h1>Detalhes do filme {id}</h1>;
}
```

`useParams()` devolve um objeto com todos os parâmetros da rota atual. Como a rota se chama `:id`, a chave no objeto também se chama `id`. Se o usuário acessar `/filme/7`, o `id` dentro do componente será a string `"7"`.

#### Analogia

Pense na rota `/filme/:id` como um formulário de rifa com um campo em branco: "Filme número \_\_\_\_". O `:id` é o espaço em branco, e o `useParams` é quem lê o que foi preenchido ali para aquele acesso específico.

### Linkando para a página de detalhe

No catálogo, cada card de filme vira um link para a própria página de detalhe:

```
<Link to={`/filme/${filme.id}`}>Ver detalhes</Link>
```

Repare no uso do **template string** (crase) para montar o endereço dinamicamente, juntando o texto fixo `/filme/` com o valor de `filme.id` — o mesmo recurso que já usamos em saudações e mensagens dinâmicas nas aulas anteriores.



### Atenção

`useParams()` sempre devolve **strings**. Se você tiver uma lista de filmes com `id` numérico e comparar `id === 1` (número), a comparação nunca vai bater. O certo é comparar com `id === "1"` (string) ou converter com `Number(id)` antes de comparar.

## 6. Navegação Programática

Mudando de página a partir do código, sem clicar num link.

Às vezes você precisa mudar de página **sem** o usuário clicar num link — por exemplo, depois de confirmar uma ação, ou ao clicar num botão "Voltar". Para isso existe o hook `useNavigate`:

```
import { useNavigate } from "react-router-dom";

function FilmeDetalhe() {
  const navigate = useNavigate();

  return (
    <div>
      <h1>Detalhes do filme</h1>
      <button onClick={() => navigate("/")}>Voltar para o catálogo</button>
    </div>
  );
}
```

`navigate("/")` manda o usuário de volta para a rota `/`, por código — o mesmo efeito de clicar num `Link`, mas disparado dentro de uma função (por exemplo, depois de validar um formulário ou concluir uma ação).

### Link × useNavigate

Use `Link` quando a navegação é o próprio conteúdo clicável (um menu, um texto "ver mais"). Use `useNavigate` quando a navegação é uma **consequência** de uma ação (enviar um formulário, clicar num botão "Voltar", finalizar uma compra).

## 7. Página 404 — A Rota Coringa

Tratando endereços que não existem.

E se o usuário digitar um endereço que não existe, tipo `/qualquer-coisa`? Sem tratamento, a tela fica em branco. A solução é uma rota **coringa**, usando `*` como caminho — ela bate com **qualquer** endereço que nenhuma outra rota tenha capturado:

```
<Routes>
  <Route path="/" element={<Home />} />
  <Route path="/sobre" element={<Sobre />} />
  <Route path="/filme/:id" element={<FilmeDetalhe />} />
  <Route path="*" element={<NotFound />} />
</Routes>
```

```
// src/pages/NotFound.jsx
export default function NotFound() {
  return <h1>404 - Página não encontrada</h1>;
}
```

### Atenção à ordem

A rota `*` deve ser sempre a **última** dentro do `Routes`. O React Router lê de cima para baixo e usa a primeira que bater — se o coringa vier primeiro, ele "rouba" todas as outras rotas, e nenhuma outra página nunca mais aparece.

## Resumo geral do que aprendemos sobre rotas

| Peça                                     | Para que serve                                                                     |
|------------------------------------------|------------------------------------------------------------------------------------|
| <code>BrowserRouter</code>               | Ativa o sistema de rotas em toda a aplicação.                                      |
| <code>Routes</code> / <code>Route</code> | Liga cada endereço ( <code>path</code> ) a um componente ( <code>element</code> ). |
| <code>Link</code>                        | Navega entre rotas sem recarregar a página.                                        |
| <code>:parametro</code>                  | Marca uma parte do caminho como dinâmica (ex.: <code>/filme/:id</code> ).          |
| <code>useParams()</code>                 | Lê o valor do parâmetro da rota atual.                                             |
| <code>useNavigate()</code>               | Muda de rota a partir do código, sem precisar de um <code>Link</code> .            |
| <code>path="*"</code>                    | Rota coringa — captura qualquer endereço não previsto (página 404).                |

Desenvolvimento Web — Guia de Estudos, Parte 2 • Rotas e Navegação com React Router • Continue praticando nos projetos feitos em aula.